

# AJAX

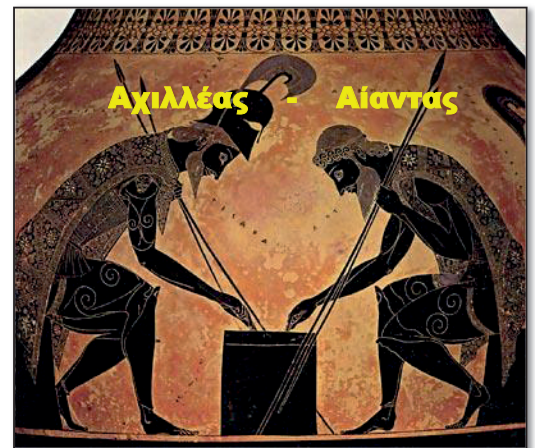


Φώτης Κόκκορας  
Πανεπιστήμιο Θεσσαλίας

v.12/05/2020

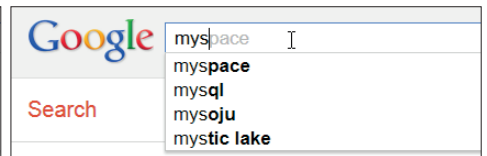
## AJAX (προφ. "έιτζας" 'eidzæks)

❖ Καμία σχέση με τα ακόλουθα:



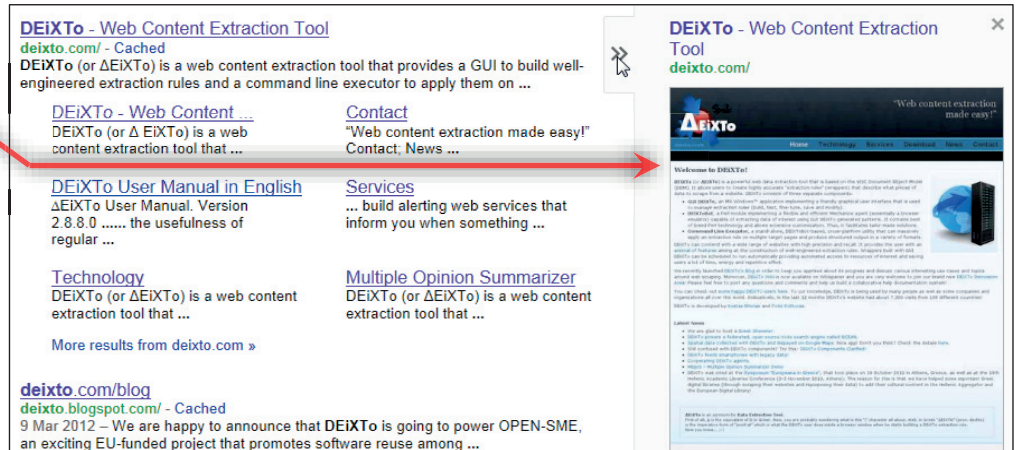
# AJAX στην πράξη:

- ❖ **Google Suggests:** Από τις πρώτες AJAX εφαρμογές ευρείας χρήσης.
  - ❑ Καθώς ο χρήστης πληκτρολογεί, η λίστα γεμίζει δυναμικά με σχετικούς όρους.
  - ❑ Το τρέχων αλφαριθμητικό στέλνεται στο server και αυτός επιστρέφει νέα λίστα με δημοφιλείς όρους σχετικούς με αυτό!



## ❖ Παράδειγμα #2:

η **προεπισκόπηση** σελίδας που εμφανίζεται στα αποτελέσματα αναζήτησης της Google.



# Βασικές Έννοιες

- ❖ Τι σημαίνει AJAX; ==> Asynchronous JavaScript and XML
- ❖ Πρόκειται για συνδυασμό υπαρχόντων τεχνολογιών του Παγκόσμιου Ιστού (JavaScript, HTML, XML, DOM, CSS) που αποσκοπεί στην κατασκευή web εφαρμογών υψηλής διαδραστικότητας (RIAs = **Rich Internet Applications**).
  - ❑ Η βασική τεχνολογία είναι η γλώσσα JavaScript.
- ❖ **Βασική Ιδέα:** Μέσω AJAX, μια web εφαρμογή (πρακτικά ο browser), μπορεί να στείλει κλήσεις GET ή POST στον web server και να λάβει από αυτόν απάντηση που θα ενσωματώσει στην τρέχουσα σελίδα, χωρίς να πλοηγηθεί σε άλλο URL.
  - ❑ Οι AJAX κλήσεις γίνονται σε επίπεδο client (browser) μέσω JavaScript και κατάλληλης υποδομής του browser (XMLHttpRequest object).
  - ❑ Η ανανέωση τμήματος της σελίδας γίνεται με JavaScript το οποίο χειρίζεται την απάντηση του web server στην AJAX κλήση που του έγινε.
  - ❑ Η απάντηση μπορεί να είναι κείμενο (text, HTML, JSON) ή XML.
- ❖ **Σε τεχνικό επίπεδο:** ο browser ανακτά δεδομένα από το server μέσω του object XMLHttpRequest (το υποστηρίζουν ΟΛΟΙ οι σύγχρονοι browsers).
  - ❑ Σε IE5 και IE6 υπάρχει ισοδύναμο ActiveX object – άρα απαιτείται ειδική μεταχείριση.
- ❖ Παρόλο που το όνομα AJAX εμπεριέχει τον όρο XML, η χρήση της XML ΔΕΝ είναι υποχρεωτική (θα μπορούσε π.χ. ο server να απαντήσει σε HTML ή σε απλό κείμενο).

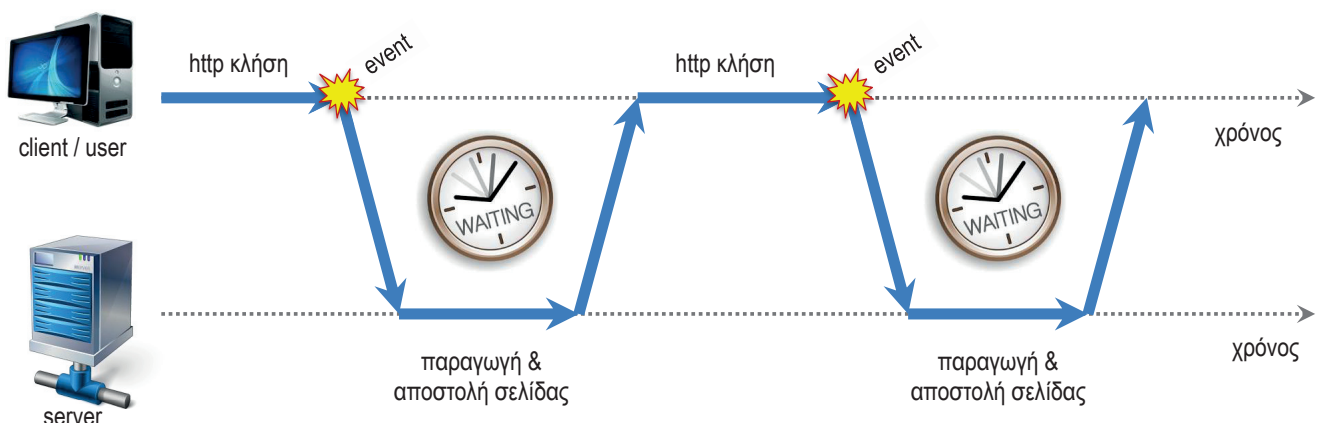


## Εξέλιξη – Ιστορικά Στοιχεία

- ❖ Ο όρος AJAX πρωτοχρησιμοποιήθηκε το 2005 (Jesse James Garrett)
- ❖ Τεχνικές για την ασύγχρονη φόρτωση περιεχομένου σε ιστοσελίδες χρησιμοποιούνται από τα μέσα της δεκαετίας του '90.
  - ❑ Τα Java applets (1995), επιτρέπουν σε client-side κώδικα να ανακτά δεδομένα από τον web-server, με "ασύγχρονο" τρόπο.
- ❖ Το 1999 η Microsoft δημιούργησε το αντικείμενο XMLHttpRequest για την ασύγχρονη μεταφορά XML (και άλλων δεδομένων κειμένου) μεταξύ server και browser.
- ❖ Το 2006, εκδίδεται από το World Wide Web Consortium (W3C) μία πρώτη πρόχειρη προδιαγραφή για το αντικείμενο XMLHttpRequest, σε μία προσπάθεια να δημιουργηθεί ένα επίσημο web standard (AJAX).
- ❖ Πιο πρόσφατη προδιαγραφή: XMLHttpRequest:
  - ❑ <http://www.w3.org/TR/XMLHttpRequest/> +

## Σύγχρονες vs. Ασύγχρονες Κλήσεις (1/2)

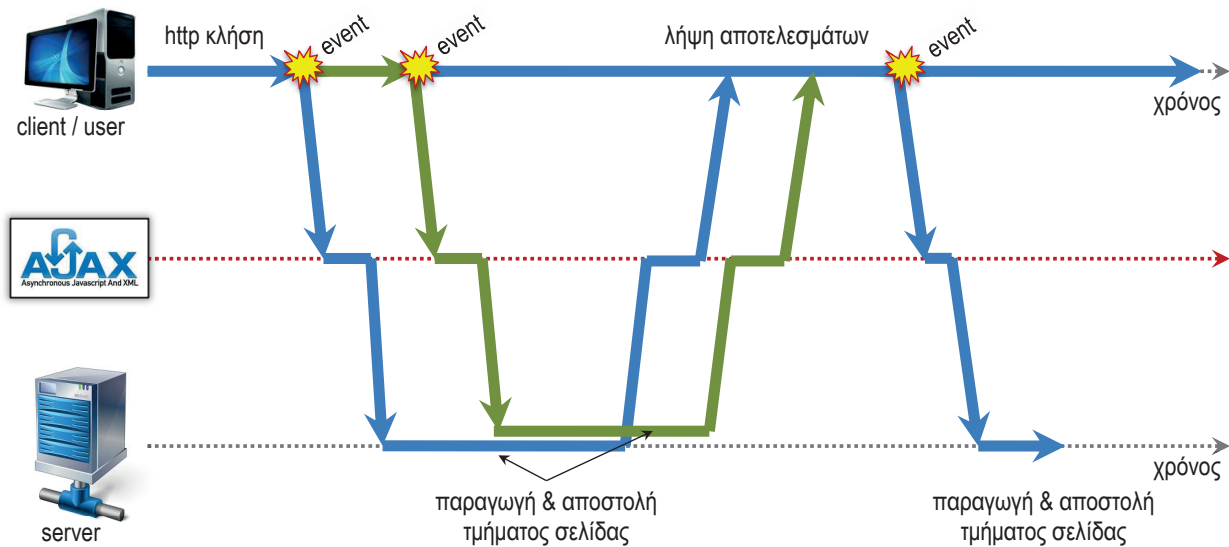
- ❖ Η JavaScript εκτελείται στην JavaScript Engine (είναι ενσωματωμένη στον browser)
- ❖ **Σύγχρονες Κλήσεις (Synchronous Calls):** η JavaScript engine περιμένει να εκτελεστούν και μετά προχωρά σε τυχόν επόμενες εντολές.
  - ❑ Δεν περιμένει βέβαια ...αιώνια! Μετά από πάροδο προκαθορισμένου χρόνου (μερικές δεκάδες seconds), αν δεν έχει εκτελεστεί η "σύγχρονη" κλήση, προκύπτει σφάλμα εκτέλεσης.



## Σύγχρονες vs. Ασύγχρονες Κλήσεις (2/2)

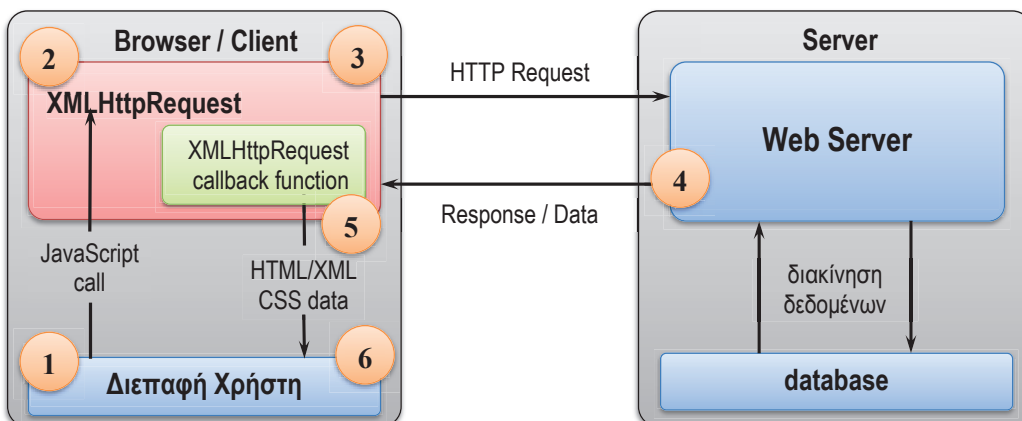
❖ **Ασύγχρονες Κλήσεις (Asynchronous Calls):** η JavaScript engine προχωρά σε τυχόν επόμενες εντολές και ολοκληρώνει τις σχετικές με "ασύγχρονες" κλήσεις εργασίες όταν οι εντολές ολοκληρωθούν.

□ Εξ'ορισμού (by default), οι AJAX κλήσεις είναι "ασύγχρονες". Μπορεί όμως να γίνουν "σύγχρονες" με κατάλληλες παραμέτρους (δε συνηθίζεται).



## Ανάλυση μιας Τυπικής Κλήσης AJAX

1. Ο χρήστης μέσω κάποιου event (π.χ. click) ενεργοποιεί ένα JavaScript.
2. Ο κώδικας του JavaScript δημιουργεί ένα XMLHttpRequest object (AJAX υποδομή).
3. Το XMLHttpRequest object προετοιμάζει και στέλνει μια HTTP κλήση στο server.
4. Ο server απαντά (με text ή XML), πιθανώς σε αλληλεπίδραση και με μια database.
5. Μόλις το XMLHttpRequest object παραλάβει την απάντηση, προκαλεί ένα event:
  - Ο προγραμματιστής έχει ορίσει από πριν μια **callback συνάρτηση** ως διαχειριστή (event handler) αυτού του event.
6. Η callback συνάρτηση εκτελείται (π.χ. ενσωματώνει την απάντηση στη σελίδα)!





# Γιατί AJAX; (άλλοι λόγοι)

- ❖ Οι web εφαρμογές...
  - ❑ ...μειονεκτούν έναντι των desktop κυρίως στη χρηστικότητα
  - ❑ ...πλεονεκτούν σε πολλά άλλα σημεία:
    - είναι προσβάσιμες από περισσότερους χρήστες
    - δεν απαιτούν συνήθως εγκατάσταση
    - είναι ευκολότερες στην υποστήριξη
    - απαιτούν στη χειρότερη περίπτωση παρόμοια προγραμματιστική προσπάθεια
- ❖ Με τις τεχνολογίες AJAX, οι web εφαρμογές μπορεί να γίνουν πλουσιότερες στο περιεχόμενο και φιλικότερες στη χρήση.
- ❖ Οι AJAX τεχνικές στην κατασκευή web εφαρμογών δημιουργούν εφαρμογές:
  - ❑ μικρότερες (προάγουν την επαναχρησιμοποίηση κώδικα)
  - ❑ γρηγορότερες (μεταφορά λιγότερων δεδομένων)
  - ❑ φιλικότερες προς τον χρήστη (αίσθηση desktop εφαρμογής – καλύτερη ροή εργασίας)
- ❖ Πετυχαίνουν τα ανωτέρω με τρόπο ανεξάρτητο από browser και υπολογιστική πλατφόρμα (λειτουργικό σύστημα).
- ❖ **Για να καταγραφεί μια AJAX κλήση σε Google Analytics απαιτείται να δηλωθεί στη Google και να γίνεται μια παράλληλη κλήση αυτή.**



# AJAX: Τι τεχνολογίες περιλαμβάνει;

- ❖ Περιλαμβάνει τις ακόλουθες τεχνολογίες:
  - ❑ Document Object Model (DOM) για δυναμική (τμηματική) απεικόνιση και αλληλεπίδραση με τα HTML στοιχεία των σελίδων
  - ❑ Το XMLHttpRequest object για ασύγχρονη επικοινωνία με το server
    - πρακτικά, η AJAX υποδομή του browser
  - ❑ XHTML/CSS ή/και XML/XSLT για την απεικόνιση των αποτελεσμάτων
    - Η ανταλλαγή δεδομένων σε XML δεν είναι υποχρεωτική, το ίδιο και η χρήση XSLT για τη διαμόρφωση του περιεχομένου που διακινείται.
      - ✓ Συνήθως ο server στέλνει κάποιο προ-μορφοποιημένο HTML περιεχόμενο (ή και απλό text), που παράγει μέσω κάποιου server-side script.
  - ❑ JavaScript για να συνδεθούν όλα τα ανωτέρω μαζί!
    - Η JavaScript δεν είναι η μόνη client-side scripting γλώσσα που χρησιμοποιείται. Άλλες γλώσσες όπως η VBScript μπορούν να παράσχουν την απαιτούμενη υποδομή.

# JavaScript Libraries/Frameworks

- ❖ Υπάρχουν αρκετές JavaScript βιβλιοθήκες που ενσωματώνουν AJAX λειτουργικότητα με τη μορφή εύχρηστων συστατικών λογισμικού (components) που επιτρέπουν τη σχετικά εύκολη δημιουργία RIAs (Rich Internet Applications).
- ❖ **Δεν είναι υποχρεωτική η χρήση τους για να κάνετε AJAX κλήσεις – απλή JavaScript αρκεί!!!**
- ❖ Ορισμένες βιβλιοθήκες που άντεξαν στο χρόνο: jQuery, ASP.net AJAX, MooTools,



<https://jquery.com>  
<https://jqueryui.com/demos/>



[AJAX Control Toolkit](#)



<https://mootools.net>

- ❖ Το σημαντικό σε αυτά τα libraries είναι ότι δίνουν εντυπωσιακά UI controls.
  - ❑ Δείτε τα demos στη home page κάθε ενός από τα παραπάνω.
- ❖ Μια καταγραφή libraries: [http://en.wikipedia.org/wiki/List\\_of\\_Ajax\\_frameworks](http://en.wikipedia.org/wiki/List_of_Ajax_frameworks)

## Παραδοσιακές vs. AJAX Λύσεις

- ❖ Για αποστολή/λήψη πληροφορίας σε/από μια database ή ένα αρχείο στο server, οι **παραδοσιακές web λύσεις** απαιτούν:
  - ❑ μια HTML φόρμα στην οποία ο χρήστης πατά το "submit" button
  - ❑ αναμονή μέχρις ότου απαντήσει ο server
  - ❑ αναμονή να παρουσιαστεί η εξ'ολοκλήρου νέα σελίδα
    - έτσι, οι παραδοσιακές web εφαρμογές είναι αργές και όχι ιδιαίτερα φιλικές στη χρήση αλλά και στον προγραμματιστή
- ❖ Στην **AJAX προσέγγιση**, κώδικας JavaScript επικοινωνεί απευθείας με το server μέσω του JavaScript XMLHttpRequest object. Μέσω αυτού:
  - ❑ Μια σελίδα μπορεί να ζητήσει και να πάρει δεδομένα από το server χωρίς να απαιτείται επαναφόρτωση ολόκληρης της σελίδας!
  - ❑ Ο χρήστης παραμένει στην ίδια σελίδα και το μόνο που συνήθως αντιλαμβάνεται είναι μια τοπική αλλαγή περιεχομένου σε κάποια περιοχή της σελίδας.
- ❖ Το **XMLHttpRequest** object υποστηρίζεται από όλους τους διαδεδομένους browsers (Internet Explorer (IE), Firefox, Chrome, Opera, Safari)
  - ❑ Οι (παμπάλαιοι πλέον) IE5 και IE6 δεν το υποστηρίζουν αλλά έχουν ισοδύναμο ActiveX object με ίδια λειτουργικότητα και τρόπο χρήσης.

## Παράδειγμα AJAX Εφαρμογής 1/2

```
<html><head>
<script type="text/javascript">
function initAJAX() {
    if (window.XMLHttpRequest) { // all modern browsers
        return xmlhttp=new XMLHttpRequest();
    } else if (window.ActiveXObject) { //for IE5, IE6
        return xmlhttp=new ActiveXObject("Microsoft.XMLHTTP");
    } else { //AJAX not supported
        alert("Your browser does not support AJAX calls!");
        return false;
    }
}

function myAJAXcall(param) {
    var ajaxObject = initAJAX();
    if (ajaxObject) {
        var url = "foo.php?param="+param;
        ajaxObject.open("GET",url,true);
        ajaxObject.send();
        ajaxObject.onreadystatechange=function() {
            if(ajaxObject.readyState==4 && ajaxObject.status==200) {
                myDIV = document.getElementById('results');
                myDIV.innerHTML = ajaxObject.responseText;
            } //if
        } //callback
    } //if
} //function
</script>
</head>
```

```
<body>
<p>
    <a href="#" onclick="myAJAXcall(1);">Link1</a>
    <a href="#" onclick="myAJAXcall(2);">Link2</a>
</p>
<div id='results'>results here</div>
</body></html>
```

*//---το αρχείο foo.php -----*

*<?php*

*//αποφυγή caching*

```
header("Cache-Control: no-cache,
must-revalidate");
```

```
header("Expires:0");
```

*//παραγωγή απάντησης*

```
if ( isset($_GET['param']) ) {
    if ($_GET['param']==1) {
        echo '<p>results for link 1!</p>';
    } elseif ($_GET['param']==2) {
        echo '<p>results for link 2!</p>';
    }
} else { echo 'missed params!'; };
```

*?>*



## Παράδειγμα AJAX Εφαρμογής 2/2

### ❖ Λειτουργικότητα παραδείγματος:

- ❑ Η σελίδα περιέχει δύο συνδέσμους και ένα div.
- ❑ Οι σύνδεσμοι δεν οδηγούν κάπου αλλά ενεργοποιούν AJAX κλήσεις προς το **foo.php**.
- ❑ Το foo.php παράγει HTML περιεχόμενο που η callback συνάρτηση ενσωματώνει μέσα στο DIV της σελίδας.

### ❖ Θα δούμε αναλυτικότερα τα ακόλουθα επιμέρους ζητήματα:

1. Πώς αρχικοποιείται η AJAX υποδομή;
2. Πώς στέλνουμε AJAX κλήσεις στο web server;
3. Πότε είναι έτοιμες οι απαντήσεις του server; Πώς/Που επιστρέφονται;
4. Πώς χειριζόμαστε τα δεδομένα που επιστρέφει ο server;
5. Πότε θα ενεργοποιηθεί όλο το παραπάνω "οικοδόμημα" που φτιάξαμε;
6. Ποιος παράγει το αποτέλεσμα στον web server;

# 1. Αρχικοποίηση AJAX Υποδομής

## XMLHttpRequest object

- ❖ Οι ασύγχρονες AJAX κλήσεις βασίζονται στο XMLHttpRequest object.
- ❖ Ποιοι browsers υποστηρίζουν το XMLHttpRequest object;
  - ❑ Υποστηρίζεται σε όλους τους πρόσφατους browsers από την εκάστοτε JavaScript engine.
  - ❑ Οι (μάλλον υπό εξαφάνιση) IE5 και IE6 χρησιμοποιούν ActiveX object και θέλουν ιδιαίτερη μεταχείριση, ευτυχώς μόνο στην αρχή των διαδικασιών.
- ❖ Ο κώδικας δημιουργίας της υποδομής για AJAX κλήσεις σε μορφή συνάρτησης:

```
<script type="text/javascript">
function initAJAX() {
    if (window.XMLHttpRequest) { //all modern browsers
        return xmlhttp=new XMLHttpRequest();
    } else if (window.ActiveXObject) { //for IE5, IE6
        return xmlhttp=new ActiveXObject("Microsoft.XMLHTTP");
    } else { //AJAX not supported
        alert("Your browser does not support AJAX calls!");
        return false;
    }
}
</script>
```



- ❖ Όλες οι σχετικές με AJAX δουλειές θα γίνουν μέσω του επιστρεφόμενου handler

# 2. Πώς στέλνουμε AJAX κλήσεις στο server;

## (χρησιμοποιούμε μεθόδους που παρέχει η AJAX υποδομή)

- ❖ **open(method, URL, mode):** προσδιορίζει τον τύπο κλήσης που θα κάνουμε
  - ❑ Οι παράμετροι με τη σειρά καθορίζουν
    - *method*: τη μέθοδο HTTP που θα χρησιμοποιηθεί (GET ή POST),
    - *URL*: το URL-στόχο (συνήθως κάποιο server-side script/σελίδα),
    - *mode*: αν το request θα γίνει ασύγχρονα:
      - ✓ true αν η κλήση γίνει "ασύγχρονα" (default τιμή)
      - ✓ false αν η κλήση γίνει "σύγχρονα"
- ❖ **send(content):** στέλνει την κλήση στον server
  - ❑ *content*: παράμετροι που θέλουμε να αποσταλούν - στις GET κλήσεις **null** ή **τίποτε**
- ❖ **datasetRequestHeader(attribute, value)**
  - ❑ Ορίζει ζεύγος **παραμέτρου-τιμής** (*attribute* = *value*) που θα σταλεί ως http header μαζί με την http κλήση.
- ❖ **setRequestHeader(header,value):** προσθέτει http headers στην κλήση
- ❖ **abort( ):** Σταματάει τη σε εξέλιξη AJAX κλήση.







# Παράδειγμα – Αποστολή AJAX κλήσης στο Server

- ❖ με **GET** (παράδειγμα σχετικό με το αρχικό μας πρόβλημα)
 

```
ajaxObject.open("GET", "foo.php", true);
ajaxObject.send(null); //null σε GET κλήσεις
```
- ❖ με **POST** (παράδειγμα άσχετο με το αρχικό μας πρόβλημα – περιλαμβάνει και πέρασμα παραμέτρων `firstname` και `lastname`).
 

```
ajaxObject.open("POST", "mytest.php", true);
ajaxObject.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
ajaxObject.send("firstname=Fotis&lastname=Kokkoras");
```
- ❖ Προσέξτε την ομοιότητα με την συμβατική εκδοχή χρήσης html φόρμας. Υπάρχει και εδώ η μέθοδος (POST ή GET), το αρχείο στο οποίο θα σταλούν τα δεδομένα (`foo.php` στο 1<sup>ο</sup> παράδειγμα, `mytest.php` στο 2<sup>ο</sup> παράδειγμα) και τα δεδομένα (`firstname` και `lastname` στο 2<sup>ο</sup> παράδειγμα).
- ❖ Η μέθοδος **send()** στέλνει το αίτημα στο server.
  - ❑ Θεωρήσαμε ότι α) η HTML σελίδα με τη φόρμα και β) το PHP αρχείο που θα ετοιμάσει την απάντηση στο αίτημα (`foo.php` και `mytest.php` για τα παραπάνω παραδείγματα), βρίσκονται στον ίδιο φάκελο στο server.



## 3α. Πότε είναι έτοιμη η απάντηση του server; (το μαθαίνουμε παρακολουθώντας ιδιότητες της AJAX υποδομής)

- ❖ **readyState:**
  - ❑ Περιέχει την κατάσταση της απάντησης του server (server's response).
  - ❑ Πιθανές τιμές της ιδιότητας **readyState** και ερμηνεία:
    - 0 Δεν έχει γίνει ακόμη αρχικοποίηση κάποιας αίτησης (request is not initialized)
    - 1 Έγινε αρχικοποίηση αίτησης (the request has been set up)
    - 2 Έγινε αποστολή της αίτησης (the request has been sent)
    - 3 Γίνεται επεξεργασία της αίτησης για διεκπεραίωση (the request is in process)
    - 4 Ολοκληρώθηκε η επεξεργασία (the request is complete)
  - ❑ Κάθε φορά που η τιμή της παραμέτρου **readyState** αλλάζει, πυροδοτείται η εκτέλεση της συνάρτησης που είναι δηλωμένη στην παράμετρο **onreadystatechange**.
- ❖ **onreadystatechange:**
  - ❑ Περιέχει τη συνάρτηση (callback) που θα λάβει και θα διαχειριστεί τα δεδομένα που στέλνει ο server ως απάντηση στην AJAX κλήση. Προσέξτε ότι η συνάρτηση ΔΕΝ έχει όνομα.
  - ❑ Η συνάρτηση θα κληθεί ΑΥΤΟΜΑΤΑ, αρκεί να έχει δηλωθεί/προγραμματιστεί! π.χ.

```
ajaxObject.onreadystatechange = function () {
    // εδώ θα μπει κώδικας που διαχειρίζεται τα δεδομένα που έστειλε ο server
}
```

#### ❖ status:

- ❑ Πρόκειται για το γνωστό **http status** ενός web server. Ενδεικτικές τιμές:
  - **200: OK**
  - **403: Forbidden**
  - **404: Page not found**
  - **401: Unauthorized** κτλ

## Σύνοψη

- ❖ Όταν το **readyState** του AJAX object είναι **4** τότε η επεξεργασία στο server έχει ολοκληρωθεί. Αν επιπλέον το και το **http status** του web server είναι **200** τότε έχουμε και την αναμενόμενη απάντηση.



- ❖ Έτσι η **callback** συνάρτηση διαμορφώνεται ως εξής:

```
ajaxObject.onreadystatechange=function() {  
    //αν ολοκληρώθηκε η επεξεργασία (4) και η κλήση είναι επιτυχής (200)  
    if (ajaxObject.readyState == 4 && ajaxObject.status==200) {  
        // Κάνε κάτι με τα δεδομένα που έστειλε ο server  
    }  
}
```



## 3β. Πώς/Που επιστρέφει ο server την απάντηση;

- ❖ Στις ιδιότητες **responseText** (ή **responseXML**) του XMLHttpRequest object
  - ❑ Περιέχει τα δεδομένα που έστειλε ο server σε μορφή κειμένου (ή XML).

## 4. Πώς χειριζόμαστε τα δεδομένα της απάντησης;

- ❖ Στο αρχικό παράδειγμα, πώς κάνουμε το server να γράψει μέσα στο DIV;  

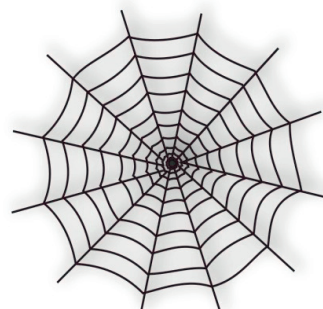
```
ajaxObject.onreadystatechange=function() {  
    if (ajaxObject.readyState == 4 && ajaxObject.status==200) {  
        document.getElementById("results").innerHTML = ajaxObject.responseText;  
    }  
}
```

## 5. Πότε θα εκτελεστεί η ajaxFunction();

- ❖ Βάση αρχικού πλάνου, η JavaScript συνάρτηση θέλουμε να εκτελεστεί αυτόματα όταν ο χρήστης κάνει click σε έναν από τους δύο συνδέσμους.

```
<p>  
    <a href="#" onclick="myAJAXcall(1);">Link1</a>  
    <a href="#" onclick="myAJAXcall(2);">Link2</a>  
</p>
```

- ❑ **onclick:** όταν γίνεται click στον σύνδεσμο.



## 6. Ποιος παράγει το αποτέλεσμα στον server;

- ❖ Στο παράδειγμα, το foo.php. Το αποτέλεσμα πρέπει να έρχεται από το ίδιο domain με τη σελίδα που κάνει τις AJAX κλήσεις και μπορεί να είναι στατικό ή δυναμικό.
  - ❑ το αποτέλεσμα βρίσκεται προσωρινά στην μεταβλητή (ιδιότητα) **responseText** (ή στη **responseXML**, ανάλογα με το τι φτιάξαμε) του XMLHttpRequest αντικειμένου
- ❖ Τι είναι καλύτερα;
  - A. να ετοιμάζεται το αποτέλεσμα πλήρως στον web server (με server side scripting / πχ php);
  - B. να στέλνονται ακατέργαστα τα δεδομένα στον client (browser) (πχ σε XML) και να τα ετοιμάζουμε εκεί με JavaScript;
- ❑ Δεν υπάρχει απόλυτη απάντηση. Αν μιλάμε για site με μεγάλη κίνηση ίσως είναι καλύτερα να μην επιβαρύνουμε τον server αλλά να "φορτώσουμε" τη δουλειά στον H/Y του κάθε χρήστη (η JavaScript τρέχει στον browser!).
  - Σκεφτείτε όμως και την περίπτωση clients με διαφορές (desktop PCs, smartphones, κτλ).

### Σύνοψη Παραδείγματος

- ❖ Κάνοντας click σε έναν από τους συνδέσμους της σελίδας,...
- ❖ ...ενεργοποιείται η AJAX κλήση προς τον server για την εκτέλεση του "foo.php"...
- ❖ ...γίνεται λήψη των αποτελεσμάτων της εκτέλεσης...
- ❖ ...και εμφάνισή τους μέσα σε HTML στοιχείο της σελίδας.
- ❖ Demo: <http://rigel.teilar.gr/sites/labs/lab10/ajax-example.html>



## Αποφυγή Caching της AJAX Κλήσης

- ❖ Ο browser κρατά τοπικά αντίγραφα των δεδομένων (σελίδων) που ζητάει ο χρήστης ώστε, υπό προϋποθέσεις, να μην τα ξαναζητά από τον server, αν ο χρήστης τα ζητήσει ξανά σχετικά σύντομα. Αυτό όμως προκαλεί πρόβλημα στις AJAX κλήσεις.
- ❖ **Client Side Λύση (hack):** Η AJAX κλήση να γίνεται κάθε φορά σε διαφορετικό url.
  - ❑ Έστω θέλουμε να καλούμε τη διεύθυνση getdata.php. Αντί αυτής, με Javascript, φτιάχνουμε το εξής:

```
var d = new Date();  
var url= "getdata.php?foo="+d;
```
  - ❑ Με αυτό τον τρόπο ο browser νομίζει ότι καλούμε διαφορετικό URL κάθε φορά καθώς η ημερομηνία-ώρα που παράγει η Date( ) είναι διαφορετική κάθε φορά.
- ❖ **Server-Side Λύση:** Χρήση κατάλληλων header εντολών που αποτρέπουν το caching.
  - ❑ Η AJAX κλήση γίνεται κανονικά στο URL που θέλουμε (χωρίς το προηγούμενο τρुक).

```
var url= "getdata.php";
```
  - ❑ Στο αρχείο getdata.php, στην αρχή του, βάζουμε τις ακόλουθες δυο γραμμές που λένε στον browser να μην κάνει cache αυτή την κλήση:

```
header("Cache-Control: no-cache, must-revalidate");  
header("Expires:0");
```
- ❖ Η server-side λύση είναι η επίσημη. Η client-side είναι προγραμματιστικό τρुक!





## Παραγωγή-Χειρισμός της Απάντησης (γενικά, όχι σε σχέση με το παράδειγμα)

- ❖ Παράδειγμα παραγωγής απλού κειμένου στο server (θα αποτελέσει το `responseText`):  
`<?php echo "Hello Ajax caller!"; ?>`
- ❖ ...και εμφάνιση του παραπάνω κειμένου στον client σε ένα alert box, μέσω της callback συνάρτησης:  
`alert("The server said: " + ajaxObject.responseText);`
- ❖ Παράδειγμα παραγωγής XML στο server:
  - ❑ Θα το δούμε αναλυτικά σε μάθημα άλλου εξαμήνου! Κάτι πολύ σημαντικό είναι η αποστολή ειδικού header που θα ενημερώνει τον client (π.χ. τον browser) ότι θα αποσταλεί XML και ότι το default (text/html).

`<?php`

`//ΠΡΟΣΟΧΗ: η παραγωγή XML output απαιτεί αποστολή ειδικού header στον browser`

`header('Content-Type: text/xml; charset=UTF-8');`

`echo '<?xml version="1.0" encoding="UTF-8"?>';`

`echo '<greeting>Hello Ajax caller!</greeting>';`

`?>`

- ❑ Αντί με `echo`, φτιάξτε καλύτερα XML έγγραφα με την `DOMDocument` class.



## Feedback στον Χρήστη (γενικά, όχι σε σχέση με το παράδειγμα)



- ❖ Συνήθως θέλουμε να δείξουμε στο χρήστη ότι εκτελείται μια λειτουργία στο υπόβαθρο κατά τη διάρκεια που περιμένουμε την απάντηση του server στην ajax κλήση.
- ❖ Η παρακάτω απλή προσέγγιση βασίζεται στην παραδοχή ότι το αποτέλεσμα της AJAX κλήσης αντικαθιστά ολοκληρωτικά το περιεχόμενο ενός div. Έστω ότι:
  - ❑ η συνάρτηση που ξεκινά την AJAX κλήση ονομάζεται `ajaxFunction()`
  - ❑ το αποτέλεσμα της AJAX κλήσης προβάλλεται στο div με `id=ajaxdiv`
  - ❑ έχουμε μια εικόνα τύπου animated GIF όπως αυτή δεξιά (π.χ. `spinner.gif`)
- ❖ Στη συνάρτηση `ajaxFunction()`, μετά την αρχικοποίηση του `XMLHttpRequest`, βάζουμε ως περιεχόμενο στο div την εικόνα `spinner.gif`
  - ❑ `document.getElementById('ajaxdiv').innerHTML = "<img src='spinner.gif' />";`
- ❖ Η εικόνα θα είναι ορατή όση ώρα αναμένουμε την απάντηση από το server. Όταν την παραλάβουμε, τότε ως γνωστό ενεργοποιείται αυτόματα η callback συνάρτηση η οποία θα γράψει ως περιεχόμενο του div την απάντηση του server!
  - ❑ `document.getElementById('ajaxdiv').innerHTML = xmlhttp.responseText;`



**Δείτε τον κώδικα στην άσκηση του σχετικού εργαστηρίου.**





# Πηγές

- ❖ [https://www.w3schools.com/js/js\\_ajax\\_intro.asp](https://www.w3schools.com/js/js_ajax_intro.asp)
- ❖ <https://www.w3schools.com/jquery/default.asp>
- ❖ <http://www.w3.org/TR/XMLHttpRequest/>

